

LEZIONE 2 – PL/SQL

Oracle PL/SQL – Logica, Loop e Gestione Errori

In questa seconda parte della lezione approfondiremo i costrutti fondamentali del linguaggio PL/SQL: la **logica condizionale** (IF e CASE), i **cicli di iterazione** (LOOP, FOR, WHILE) e la **gestione delle eccezioni**. Ogni concetto sarà accompagnato da esempi pratici eseguibili su Oracle Database.



Ripasso: Blocchi, Variabili e SELECT INTO

Struttura del blocco PL/SQL

Ogni blocco PL/SQL segue la struttura standard con tre sezioni opzionali. La sezione `DECLARE` contiene le variabili, `BEGIN` racchiude la logica eseguibile e `EXCEPTION` gestisce gli errori.

```
DECLARE
  v_nome VARCHAR2(50);
  v_sal  NUMBER;
BEGIN
  -- istruzioni
EXCEPTION
  -- gestione errori
END;
```

Variabili e tipi di dato

Le variabili vengono dichiarate nella sezione `DECLARE` con un nome (convenzione: prefisso `v_`), un tipo e un valore iniziale opzionale.

- `v_salary NUMBER := 5000;`
- `v_name VARCHAR2(100) := 'Rossi';`
- `v_date DATE := SYSDATE;`
- `v_flag BOOLEAN := TRUE;`

SELECT INTO

Il costrutto `SELECT INTO` estrae una singola riga da una tabella e la assegna a una o più variabili. Solleva `NO_DATA_FOUND` se non trova righe e `TOO_MANY_ROWS` se ne trova più di una.

```
SELECT salary, first_name
INTO v_sal, v_name
FROM employees
WHERE employee_id = 101;
```

IF / ELSE e CASE

Il PL/SQL offre due strumenti principali per implementare la logica condizionale: il costrutto `IF-THEN-ELSE` e l'istruzione `CASE`. La scelta dipende dalla complessità della condizione da valutare.

IF – THEN – ELSIF – ELSE

Il costrutto `IF` valuta una condizione booleana. Supporta ramificazioni multiple con `ELSIF` e un ramo finale `ELSE`.

```
IF v_salary > 5000 THEN
    v_bonus := 1000;
ELSIF v_salary > 3000 THEN
    v_bonus := 500;
ELSE
    v_bonus := 200;
END IF;
```

Ogni ramo `ELSIF` viene valutato solo se tutte le condizioni precedenti sono false. Il blocco termina sempre con `END IF;`.

CASE – Alternativa più leggibile

Quando si confronta la stessa variabile con più valori, `CASE` è più conciso e leggibile di una catena di `ELSIF`.

```
CASE v_grado
    WHEN 'A' THEN v_premio := 500;
    WHEN 'B' THEN v_premio := 300;
    WHEN 'C' THEN v_premio := 100;
    ELSE          v_premio := 0;
END CASE;
```

Esiste anche la forma `CASE WHEN` per condizioni complesse non basate su uguaglianza diretta. In generale, preferire `CASE` quando si hanno più di 3 rami condizionali sullo stesso valore.

Esempio: Logica IF su Stipendio

Vediamo un esempio concreto di applicazione della logica IF per calcolare un aumento stipendiale differenziato in base alla fascia salariale del dipendente.

```
DECLARE
  v_salary    NUMBER;
  v_new_salary NUMBER;
BEGIN
  -- Recupera lo stipendio corrente
  SELECT salary
  INTO   v_salary
  FROM   employees
  WHERE  employee_id = 101;

  -- Applica aumento differenziato
  IF v_salary > 3000 THEN
    v_new_salary := v_salary * 1.10; -- +10%
  ELSIF v_salary > 2000 THEN
    v_new_salary := v_salary * 1.07; -- +7%
  ELSE
    v_new_salary := v_salary * 1.05; -- +5%
  END IF;

  DBMS_OUTPUT.PUT_LINE('Nuovo stipendio: ' || v_new_salary);
END;
```

Cosa succede passo per passo

1. Il `SELECT INTO` recupera lo stipendio del dipendente con ID 101
2. La prima condizione verifica se lo stipendio è superiore a 3000
3. In caso affermativo, applica un aumento del **10%**
4. Altrimenti verifica la seconda fascia e applica il **7%**
5. Per gli stipendi più bassi, applica il **5%**
6. Il risultato viene stampato con `DBMS_OUTPUT.PUT_LINE`

Nota sull'operatore di assegnazione

In PL/SQL l'operatore di assegnazione è `:=` (due punti seguiti da due punti), non il semplice `=` usato in SQL. Questo è un errore frequente per chi proviene da altri linguaggi.



Consiglio: Usare sempre parentesi tonde nelle espressioni aritmetiche per evitare ambiguità di precedenza degli operatori.

LOOP Base e LOOP con EXIT

Il ciclo `LOOP` è la forma più elementare di iterazione in PL/SQL. Ripete un blocco di istruzioni indefinitamente fino a quando non viene incontrata un'istruzione di uscita.

LOOP – Ripetizione semplice

Il ciclo `LOOP` senza condizioni esegue le istruzioni al suo interno in modo continuo. Da solo non è utile: richiede un meccanismo di uscita esplicito.

```
DECLARE
  v_n NUMBER := 1;
BEGIN
  LOOP
    DBMS_OUTPUT.PUT_LINE(v_n);
    v_n := v_n + 1;
    -- Senza EXIT questo ciclo non termina mai!
  END LOOP;
END;
```

Un ciclo `LOOP` senza uscita genera un **loop infinito**, che in produzione può bloccare sessioni e consumare risorse del server.

LOOP con EXIT – Controllo dell'uscita

L'istruzione `EXIT WHEN` definisce la condizione di terminazione del ciclo. Quando la condizione diventa vera, il ciclo termina e l'esecuzione prosegue dopo `END LOOP`;

```
DECLARE
  v_n NUMBER := 1;
BEGIN
  LOOP
    DBMS_OUTPUT.PUT_LINE(v_n);
    v_n := v_n + 1;
    EXIT WHEN v_n > 10; -- esce quando v_n supera 10
  END LOOP;
END;
```

Esiste anche la forma `EXIT`; senza condizione, usata all'interno di blocchi `IF` per uscite condizionate più complesse. È buona pratica inserire sempre un meccanismo di uscita esplicito.

FOR LOOP – Iterazione Semplice e Controllata

Il `FOR LOOP` è il ciclo più utilizzato in PL/SQL quando si conosce in anticipo il numero di iterazioni. La variabile di ciclo viene dichiarata implicitamente e gestita automaticamente.

```
DECLARE
BEGIN
  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE('Iterazione: ' || i);
  END LOOP;
END;
```

Caratteristiche del FOR LOOP

- La variabile `i` è dichiarata **implicitamente** — non serve in `DECLARE`
- L'intervallo `1..5` definisce inizio e fine (inclusivi)
- L'incremento è sempre di **+1** automatico
- La variabile è **sola lettura**: non può essere modificata nel corpo
- Il ciclo termina automaticamente al raggiungimento del limite

FOR LOOP con REVERSE

Aggiungendo la parola chiave `REVERSE` il ciclo scorre in ordine decrescente, dal valore più alto a quello più basso.

```
BEGIN
  FOR i IN REVERSE 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE(i); -- stampa: 5, 4, 3, 2, 1
  END LOOP;
END;
```

FOR LOOP numerico vs cursor FOR

Esistono due varianti: il **FOR numerico** (visto sopra) e il **cursor FOR** che itera sui risultati di una query. Quest'ultimo sarà trattato nelle lezioni avanzate.

WHILE LOOP – Iterazione Condizionata

Il `WHILE LOOP` ripete le istruzioni finché una condizione booleana rimane vera. A differenza del `FOR LOOP`, non ha un contatore implicito: la condizione di continuazione viene valutata **prima** di ogni iterazione.

Sintassi e comportamento

```
DECLARE
  v_n NUMBER := 1;
BEGIN
  WHILE v_n <= 10 LOOP
    DBMS_OUTPUT.PUT_LINE(v_n);
    v_n := v_n + 1;
  END LOOP;
END;
```

Se la condizione è falsa **fin dall'inizio**, il corpo del ciclo non viene mai eseguito. Questo lo distingue dal `LOOP` base che esegue almeno una volta prima di verificare l'uscita.

Confronto tra i tre cicli

LOOP + EXIT

Massima flessibilità. Uscita controllata manualmente con `EXIT WHEN`. Ideale per condizioni complesse.

FOR LOOP

Iterazione su intervallo noto. Contatore automatico. Più conciso e meno soggetto a errori.

WHILE LOOP

Condizione dinamica. Ideale quando il numero di iterazioni non è noto a priori.

  **Attenzione:** Nel `WHILE LOOP` assicurarsi sempre che la condizione possa diventare falsa, altrimenti si crea un loop infinito.

EXCEPTION – Gestione delle Eccezioni

La sezione `EXCEPTION` consente di intercettare e gestire gli errori runtime in modo controllato, evitando che il programma termini in modo anomalo. Oracle fornisce eccezioni predefinite e permette di definirne di personalizzate.

NO_DATA_FOUND

Sollevata quando un `SELECT INTO` non restituisce alcuna riga. È l'eccezione più comune nei blocchi PL/SQL che leggono dati da tabelle.

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Nessun
dato trovato');
```

TOO_MANY_ROWS

Sollevata quando un `SELECT INTO` restituisce più di una riga. Indica che la query non è sufficientemente selettiva o che i dati sono duplicati.

```
WHEN TOO_MANY_ROWS THEN
  DBMS_OUTPUT.PUT_LINE('Più righe
del previsto');
```

WHEN OTHERS

Cattura **tutte le altre eccezioni** non gestite esplicitamente. Deve essere sempre l'ultima clausola `WHEN`. Utile per logging e debugging.

```
WHEN OTHERS THEN
  DBMS_OUTPUT.PUT_LINE('Errore: '
|| SQLERRM);
```

 **Best Practice:** Gestire sempre `NO_DATA_FOUND` e `TOO_MANY_ROWS` esplicitamente. Usare `WHEN OTHERS` come rete di sicurezza finale, mai come unica gestione errori.

SQL dentro PL/SQL – I Comandi DML

Una delle caratteristiche più potenti del PL/SQL è la possibilità di eseguire direttamente comandi SQL all'interno dei blocchi. Questo permette di combinare la potenza del linguaggio procedurale con quella del linguaggio di interrogazione.



SELECT INTO

Estrae una singola riga e la assegna a variabili. Solleva eccezioni se il risultato non è esattamente una riga.

```
SELECT salary INTO
v_sal
FROM employees
WHERE employee_id
= 101;
```



UPDATE

Modifica i dati di una tabella. La clausola `WHERE` è fondamentale per limitare le righe interessate.

```
UPDATE employees
SET salary =
salary * 1.05
WHERE
department_id =
50;
```



INSERT

Inserisce una nuova riga in una tabella. I valori possono provenire da variabili PL/SQL o espressioni.

```
INSERT INTO
employees
(employee_id,
first_name,
salary)
VALUES (v_id,
v_name, v_sal);
```



DELETE

Rimuove righe da una tabella. Anche qui la clausola `WHERE` è critica per evitare cancellazioni accidentali.

```
DELETE FROM
employees
WHERE employee_id
= v_id;
```

✓ **Collegamento con SQL:** Tutti i comandi DML visti in SQL funzionano identici in PL/SQL. La differenza è che in PL/SQL i valori possono essere variabili e i risultati possono essere gestiti proceduralmente.

Esempio Integrato: SELECT · IF · LOOP · EXCEPTION

Un esempio che combina tutti i costrutti visti nella lezione: lettura dati, logica condizionale, ciclo di iterazione e gestione delle eccezioni in un unico blocco PL/SQL completo e funzionante.

```
DECLARE
  v_salary  NUMBER;
  v_bonus   NUMBER;
  v_dept    NUMBER := 50;
BEGIN
  -- 1. SELECT INTO: legge lo stipendio
  BEGIN
    SELECT salary
    INTO   v_salary
    FROM   employees
    WHERE  employee_id = 101;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      v_salary := 0;
      DBMS_OUTPUT.PUT_LINE('Dipendente non trovato');
    WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE('Errore: dati duplicati');
      RAISE;
  END;

  -- 2. IF: calcola il bonus
  IF v_salary > 3000 THEN
    v_bonus := v_salary * 0.10;
  ELSIF v_salary > 0 THEN
    v_bonus := v_salary * 0.05;
  ELSE
    v_bonus := 0;
  END IF;

  -- 3. LOOP: stampa dettaglio
  FOR i IN 1..3 LOOP
    DBMS_OUTPUT.PUT_LINE('Anno ' || i ||
                        ' - Bonus: ' || v_bonus * i);
  END LOOP;

  EXCEPTION
    -- 4. EXCEPTION: gestione errori globali
    WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Errore generico: ' || SQLERRM);
  END;
```

Struttura del blocco

1. **SELECT INTO** annidato con gestione locale delle eccezioni
2. **IF-ELSIF-ELSE** per il calcolo del bonus differenziato
3. **FOR LOOP** per simulare un report pluriennale
4. **WHEN OTHERS** come gestione errori globale finale

Concetti chiave della lezione

- Logica condizionale: **IF / CASE**
- Cicli: **LOOP / FOR / WHILE**
- Eccezioni: **NO_DATA_FOUND, TOO_MANY_ROWS, OTHERS**
- SQL in PL/SQL: **SELECT, UPDATE, INSERT, DELETE**
- Blocchi annidati per gestione errori locale



Prossima lezione: Cursori espliciti, procedure e funzioni memorizzate nel database.